

Git wird greifbar.

Änderungen verstehen.
Gemeinsam weiterbauen.

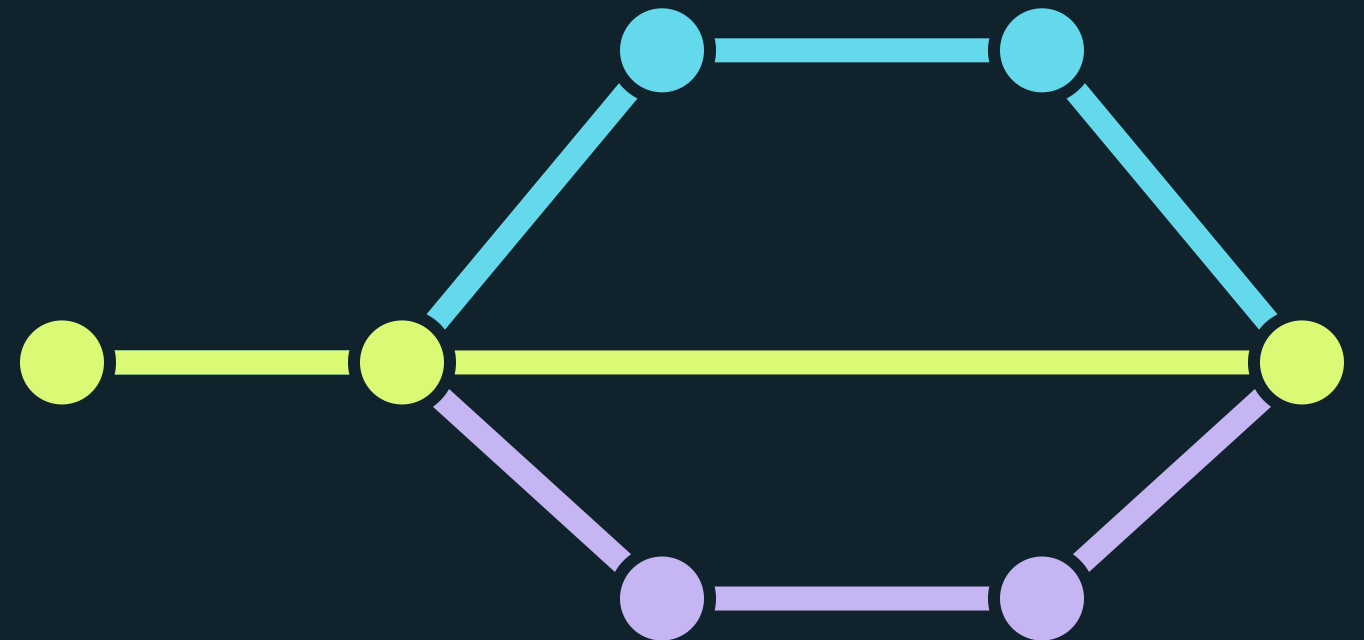
10 Personen

5 Teams

120 Minuten

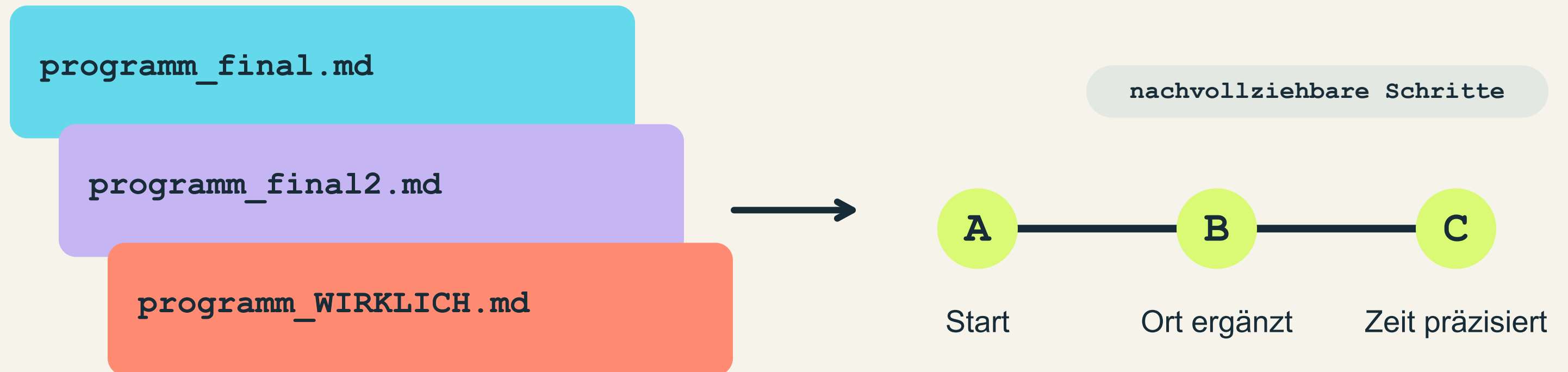
BRANCH → PR → REVIEW → MERGE

Ein Dojo für die Zusammenarbeit.



Welche Version gilt?

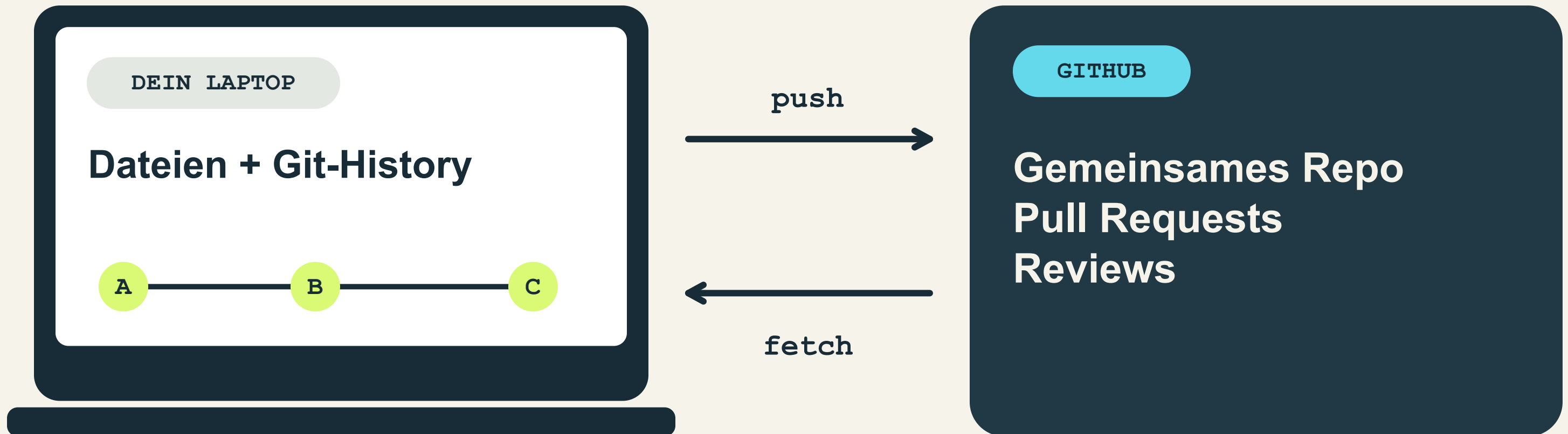
Aus Dateichaos wird eine nachvollziehbare Geschichte.



Jeder Schritt hat einen Stand, eine Begründung und einen Platz in der History.

Git ist lokal. GitHub verbindet.

Ein Commit funktioniert auch ohne Internet.



Commit $\neq$ Push. Ein PR ist der gemeinsame Raum für Prüfung und Integration.

Ändern. Auswählen. Festhalten.

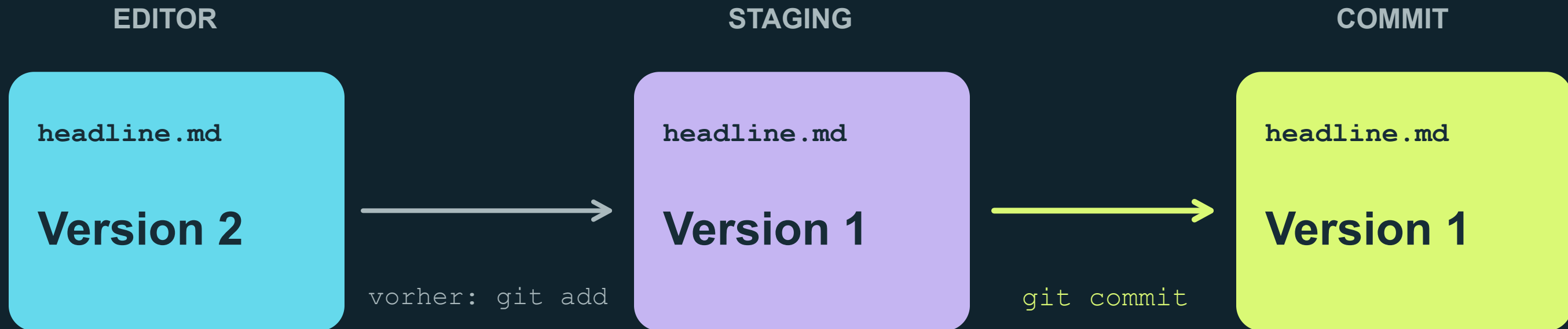
Drei Bereiche auf deinem Rechner. Drei verschiedene Zustände.



Erst den Diff lesen. Dann gezielt stagen. Dann committen.

Der Commit nimmt die Auswahl.

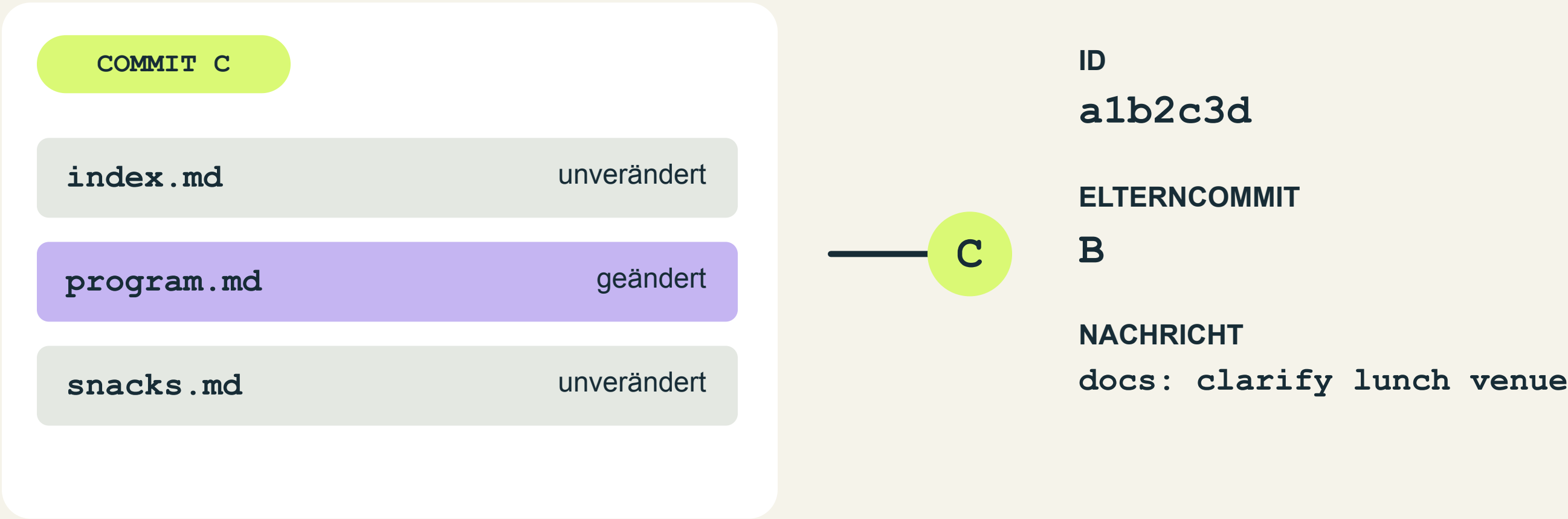
Nach `git add` weiterbearbeitet? Diese Änderung ist noch nicht ausgewählt.



Nochmals `git add` ausführen, wenn Version 2 ebenfalls in den Commit soll.

Ein Commit hält einen Stand fest.

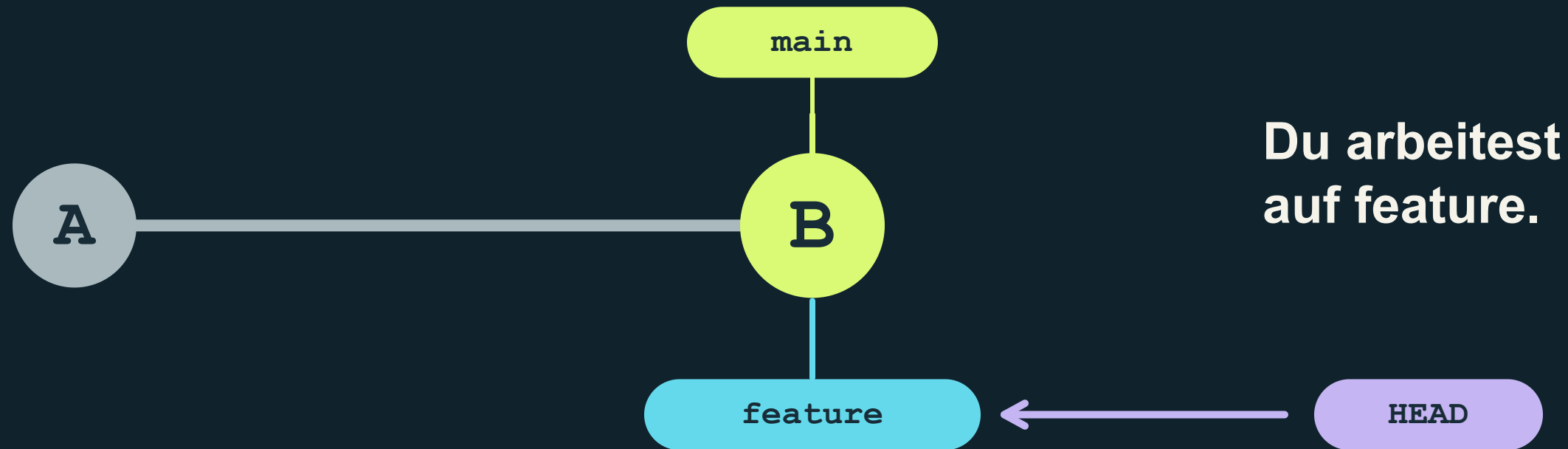
Der Diff zeigt die Änderung. Der Commit verweist auf den Projektstand.



Ein guter Commit ist ein verständlicher Schritt mit einer hilfreichen Nachricht.

Ein Branch ist ein Namensschild.

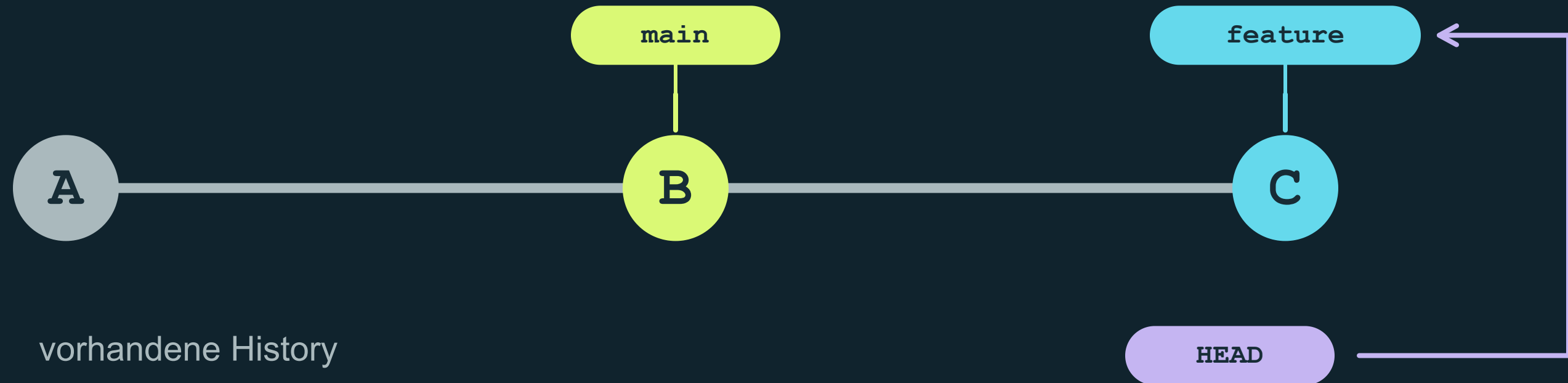
Zwei Branch-Namen können auf denselben Commit zeigen.



`git switch -c feature` → neuer Name am aktuellen Commit

Nur dein Branch bewegt sich.

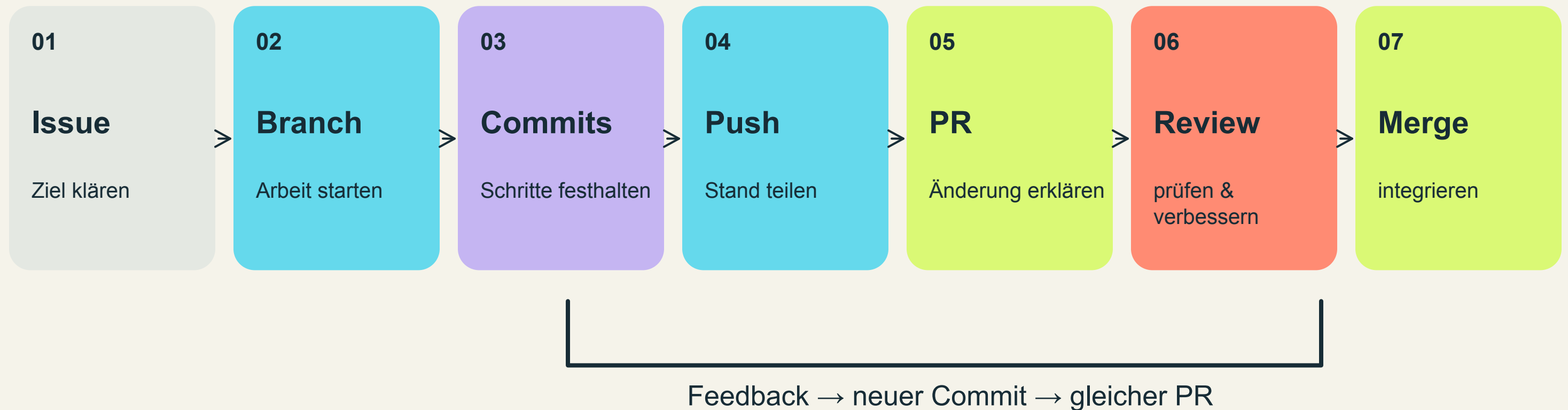
Ein neuer Commit auf feature lässt main unverändert.



Der Branch folgt deinen neuen Commits. Andere Branches bleiben stehen.

So wird aus einer Idee Teamarbeit.

Kleine Änderungen. Sichtbarer Kontext. Eine bewusste Freigabe.



main bleibt der gemeinsam geprüfte Stand.

Euer erster Pull Request.

Ein Auftrag pro Team. Nach dem ersten Commit wechselt das Keyboard.

20

MINUTEN

- 1 Startseite
- 2 Programm
- 3 Snacks
- 4 Kontakte
- 5 Status

- 1 Branch erstellen
- 2 Zwei sinnvolle Commits
- 3 Push + PR beschreiben

Checkpoint: 5 offene PRs. Ziel und Prüfung sind beschrieben. Noch nicht mergen.

Ein PR macht Entscheidungen sichtbar.

Der Diff zeigt das Was. Die Diskussion klärt das Warum.

program.md

- 16:30 Filmwerkstatt

+ 16:30 Filmwerkstatt, Raum B

REVIEW

„Wo findet der neue Slot statt?
Bitte den Ort ergänzen.“

prüfen

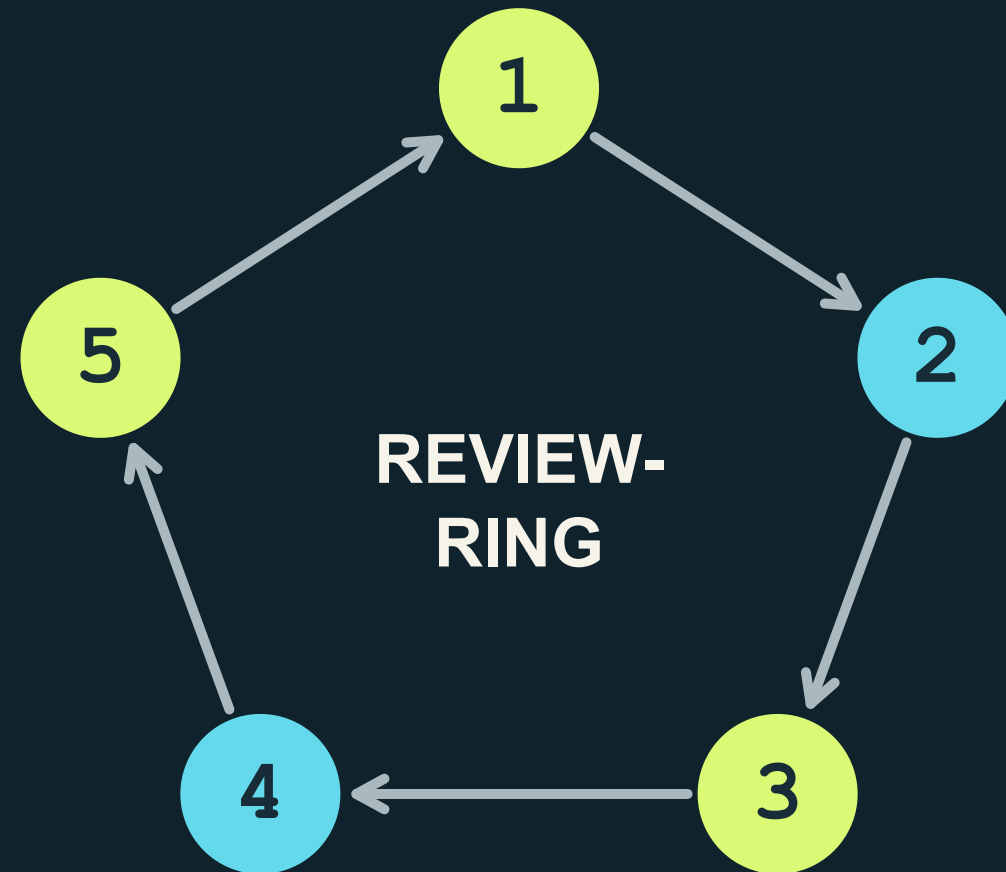


Approve

Rückmeldung → Nachbesserung → erneute Prüfung → Freigabe

Jetzt prüft das nächste Team.

Beide Personen schreiben einen sichtbaren Review-Beitrag.



1 Stärke

konkret begründen

1 Verbesserung

Wirkung erklären

Finales Review

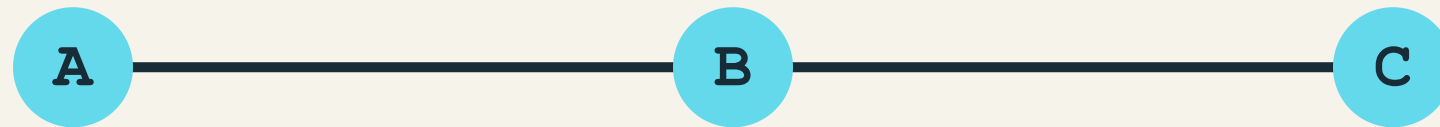
nachbessern → prüfen → mergen

Entwürfe absenden. Erst nach finalem Review: Create a merge commit.

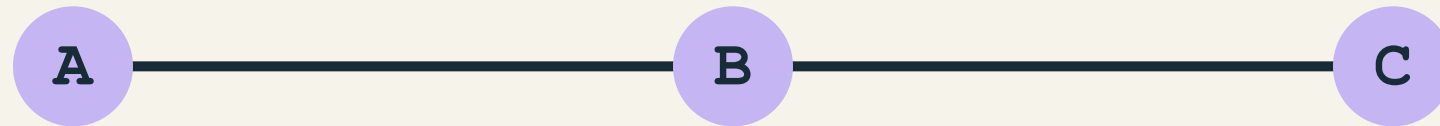
Holen ist noch nicht integrieren.

Zustand nach git fetch origin: Dein Arbeitsbranch kann noch zurückliegen.

Server: main



Lokal: origin/main



Lokal: main

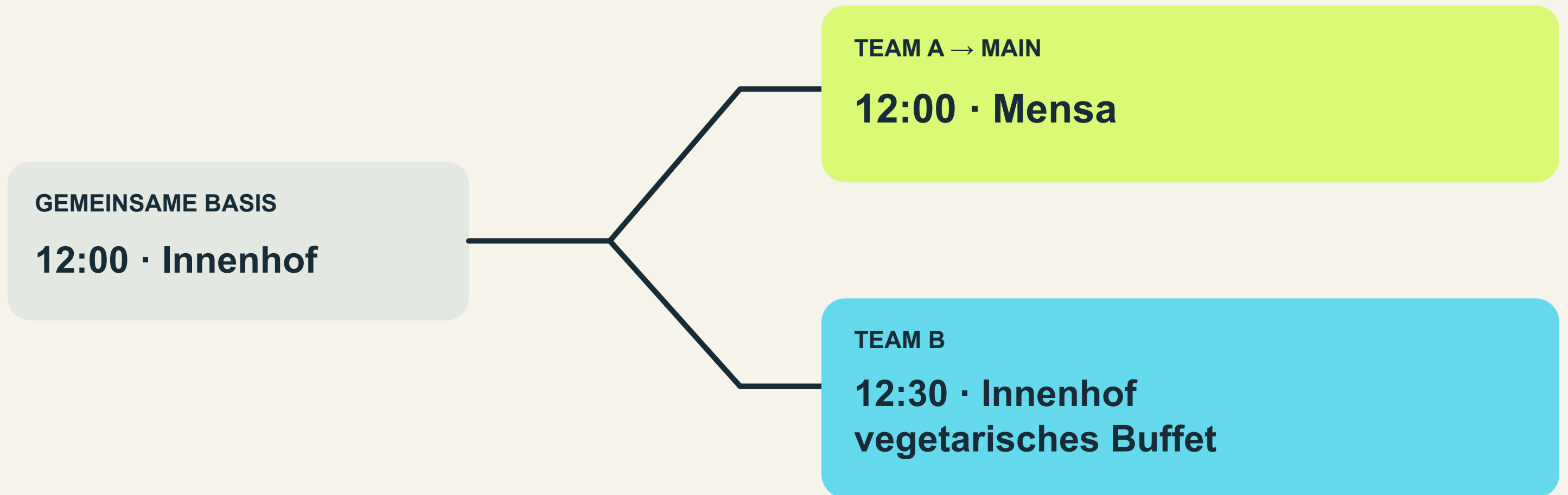


noch nicht integriert

fetch holt den Stand. pull holt und versucht zusätzlich zu integrieren.

Gleicher Start. Zwei Entscheidungen.

Beide Branches haben dieselbe Zeile unterschiedlich verändert.



Git erkennt den Textkonflikt. Ihr klärt die gültige Information für die Gäste.

Git stoppt. Ihr entscheidet.

Im lokalen Konfliktlabor, auf team-b: git merge main

<<<<<<< HEAD

- 12:30 Lunch im Innenhof mit vegetarischem Buffet

=====

- 12:00 Lunch in der Mensa

>>>>>>> main

VORGABEN

12:30

Mensa

vegetarisch

Die Marker zeigen Varianten. Die fachlich richtige Endfassung bestimmt ihr.

Baut die gültige Lunch-Zeile.

Jede Person löst einen echten Konflikt. Danach erklärt ihr euch die Entscheidung.

25

MINUTEN GESAMT

01

Lesen

Status + beide Varianten

02

Entscheiden

Zeit, Ort und Angebot verbinden

03

Abschliessen

prüfen → add → commit

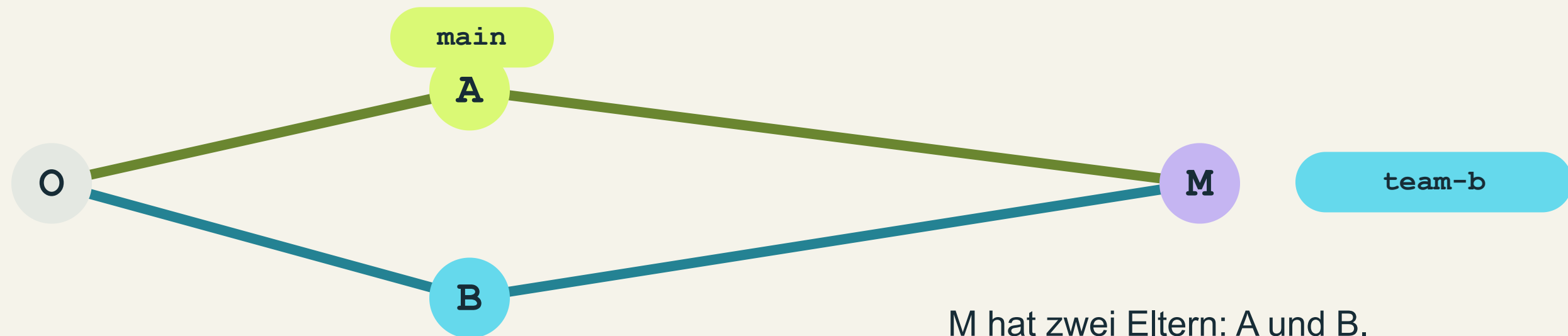
Fertig: keine Marker, alle drei Vorgaben erfüllt, sauberer git status.

Drei Anforderungen. Eine Lösung.

Eine gültige Endfassung enthält die Informationen aus beiden Änderungen.

12:30 · Lunch in der Mensa

mit vegetarischem Buffet

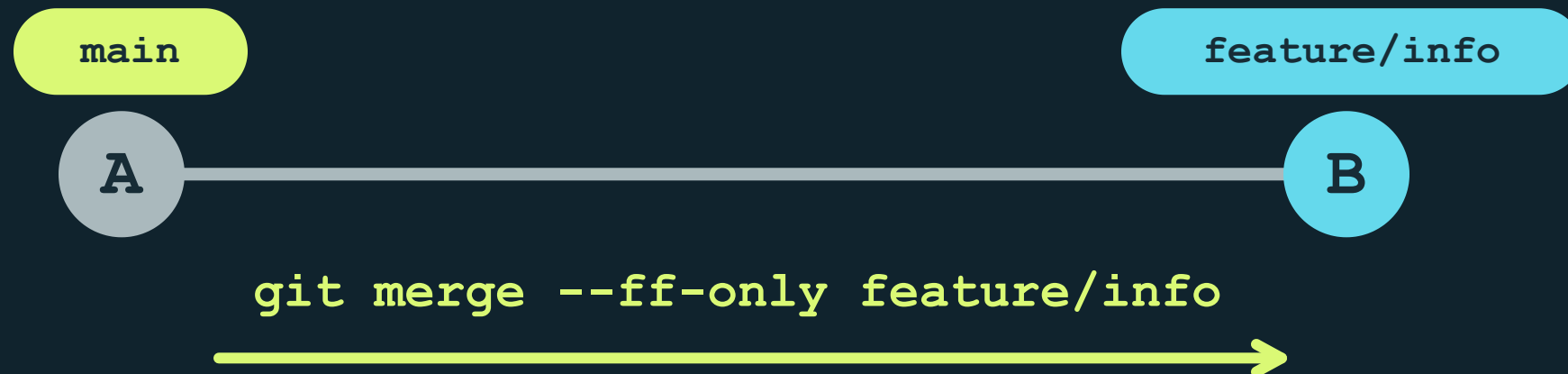


Konfliktfrei heisst: Git konnte zusammenführen. Richtig heisst: Ihr habt geprüft.

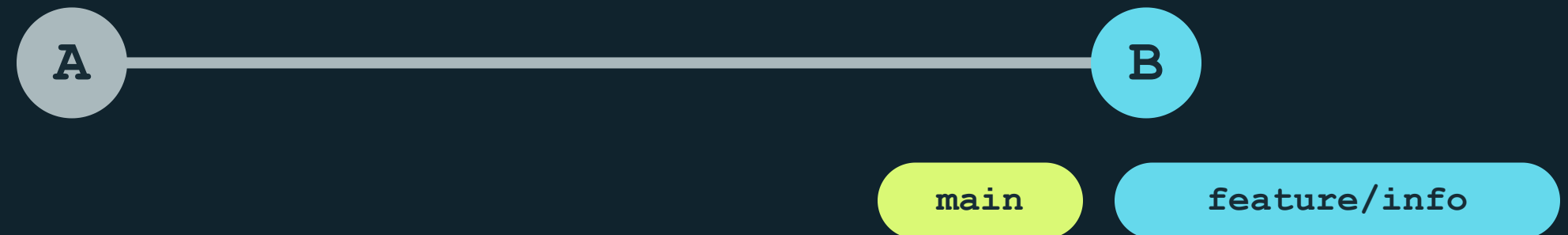
Fast-Forward: Das Schild rückt nach.

Die Commits bleiben dieselben. Nur der Zielbranch bewegt sich.

VORHER



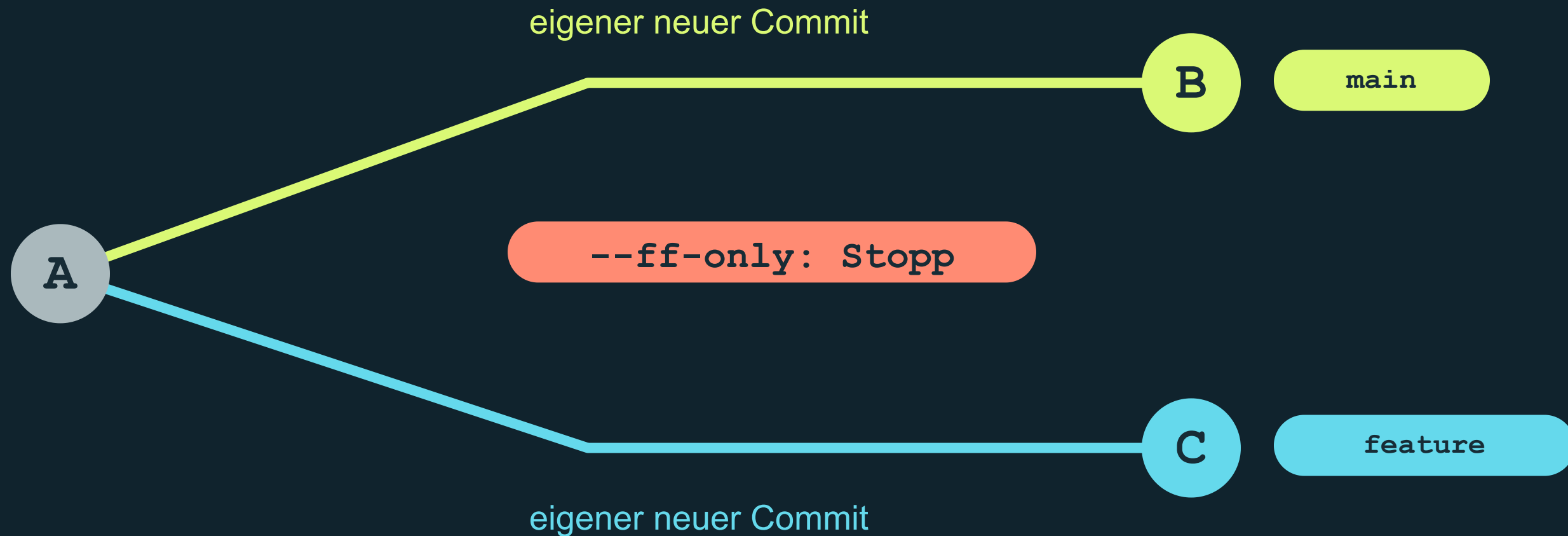
NACHHER



Kein neuer Commit. main steht jetzt auf dem vorhandenen Commit B.

Beide Linien sind weitergelaufen.

Fast-Forward ist hier unmöglich – auch ohne Dateikonflikt.

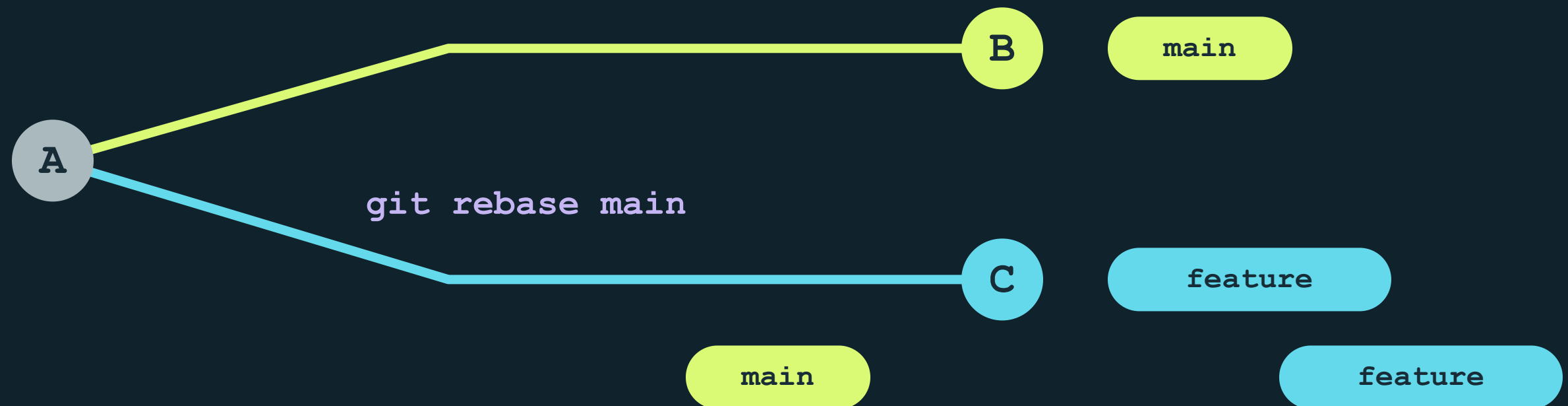


Divergenz beschreibt die History. Ein Konflikt betrifft die Zusammenführung.

Rebase: gleiche Änderung, neue Basis.

Der Feature-Branch wird neu aufgebaut. main bleibt noch stehen.

VORHER



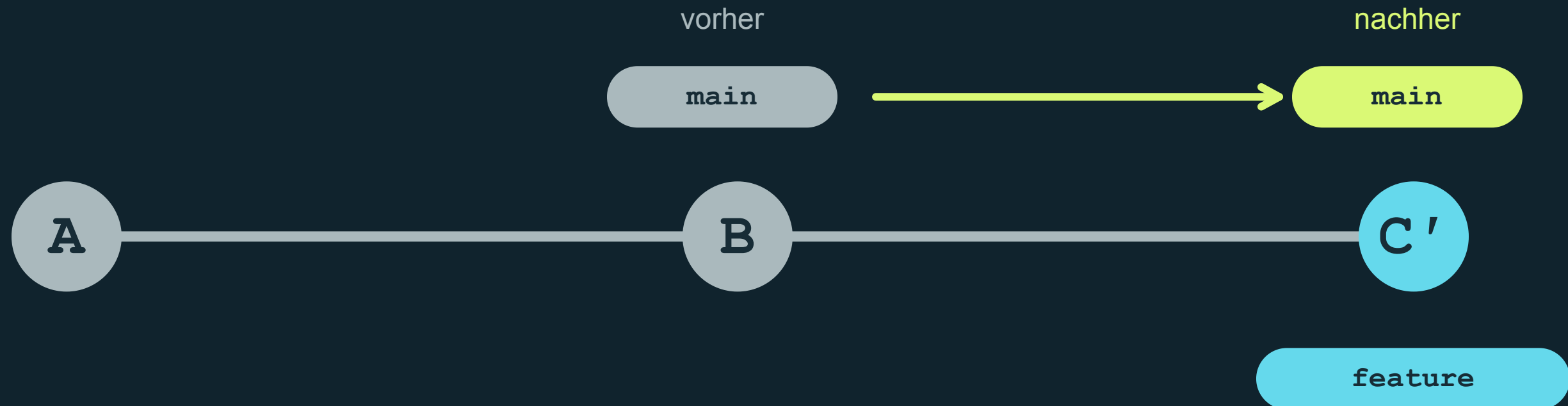
NACHHER



C → C': neue Basis, neuer Elterncommit, neue Commit-ID.

Jetzt kann main folgen.

Zuerst Rebase auf feature. Danach Fast-Forward auf main.



```
1 git switch main
```

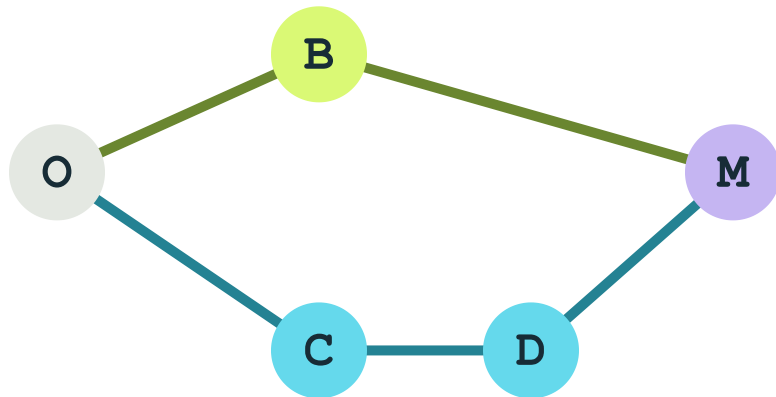
```
2 git merge --ff-only feature/volunteers
```

Das Fast-Forward erzeugt keinen weiteren Commit.

Drei Wege. Drei Geschichten.

Entscheidet nach Zusammenarbeit und gewünschter Nachvollziehbarkeit.

Merge-Commit



zwei Linien verbinden

Rebase + FF



Änderungen neu aufbauen

Squash



Schritte zusammenfassen

Gemeinsam genutzte History nur nach Absprache umschreiben.

Lest die History.

In den lokalen Laboren: vorhersagen → ausführen → Graph vergleichen.

01

Fast-Forward

Kein neuer Commit.



02

Rebase

Neue Basis. Neue ID.



03

Squash

Bonus: 3 → 1 Commit.



Ohne Remote. Kein Push nötig. Squash nur bei zusätzlicher Zeit.

Was passiert als Nächstes?

Erst selbst entscheiden. Dann im Paar begründen.

01

add → edit → commit

Welcher Stand wird gespeichert?

02

fetch ist fertig

Ist main damit schon aktuell?

03

Merge ohne Konflikt

Ist das Ergebnis deshalb richtig?

Eine gute Erklärung beschreibt den Zustand vor und nach dem Befehl.

Erst verstehen. Dann integrieren.

Welche drei Regeln nehmt ihr in eure nächste Zusammenarbeit mit?

01

Zustand lesen

Repository
Branch
Status

02

Kleine Schritte

klare Commits
verständliche PRs

03

Gemeinsam prüfen

Feedback
Nachbesserung
Freigabe

github.com/Witzelfitz/git-workflow-dojo